Report

1. Overview

This project implements a parallel word count system using a Python gRPC server and a C++ MPI backend. The Python layer provides a simple and reliable API, while MPI handles fast parallel computation.

The system has three main components:

- **Client:** Sends requests
- **gRPC Server:** Receives requests
- **MPI Engine:** Uses multiple processes to count words in parallel

Service Setup:

- The service interface is defined using **Protocol Buffers**.
- A **Python gRPC server** implements this interface.
- The server waits for client requests and processes them one at a time.

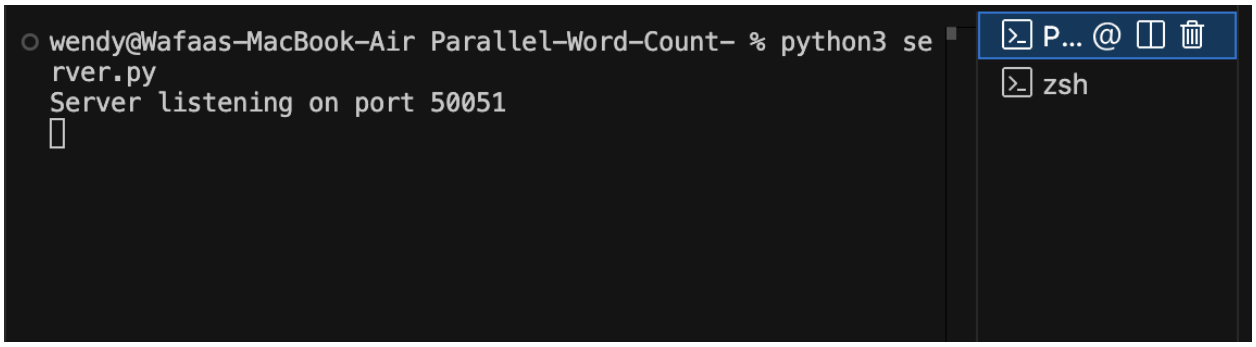
How the Code Works

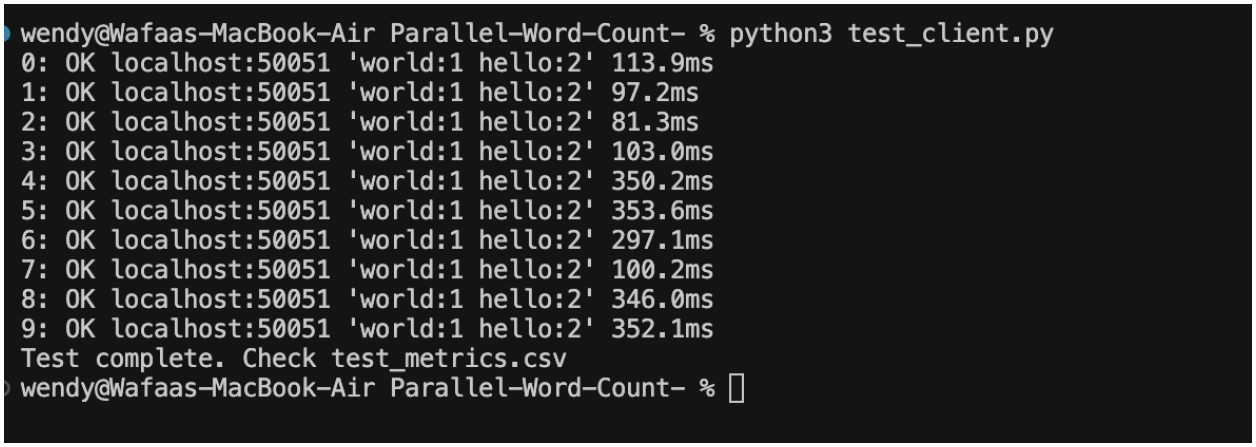
When the client sends a request, the server does the following:

- Receives the request through gRPC.
- Reads the input text and a unique request_id.
- Checks if this request_id was already processed:
    - If yes, the server returns the saved result.
    - If not, the server continues to run the algorithm.

Running the Algorithm

- The server writes the input text to a temporary file.
- It starts the computation process using an external command.
- After the computation finishes, the server reads the result.
- The result is saved in memory and sent back to the client.





Request Index (0)

This value denotes the sequence number of the request. The client issues multiple requests , and this index identifies the current request within that sequence.

Request Status (OK)

The status indicates that the request was successfully processed by the server without any r communication errors.

Server Endpoint (localhost:50051)

This field specifies the network address and port of the gRPC server that handled the request.

Returned Result
represents the word count output produced by the service.

End-to-End Latency

This value measures the total round-trip time of the request, encompassing client-side transmission, server-side processing, and response delivery.

Performance Data Collection:

the client logs performance metrics for each request to a file named test_metrics.csv.
These recorded metrics enable quantitative analysis of system performance

2. System Architecture

The architecture follows a **replicated service model**, consisting of:

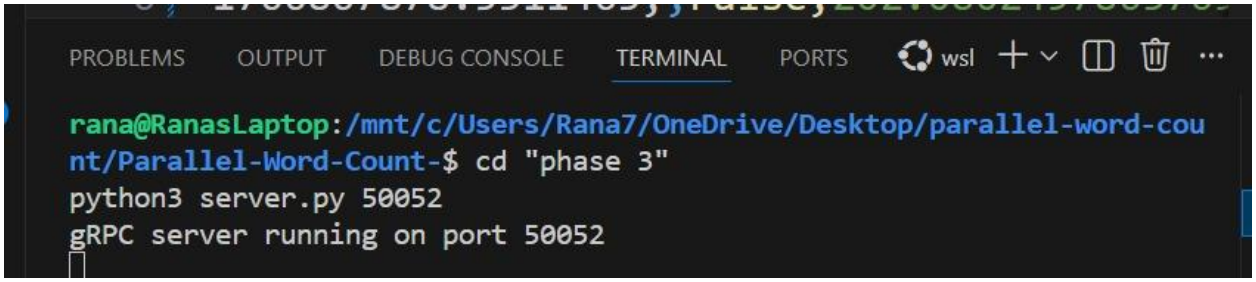
- **Two independent gRPC server replicas**:
    - Replica 1 running on port 50051
    - Replica 2 running on port 50052
- **A fault-aware gRPC client** that:
    - Maintains a list of replicas
    - Retries failed requests
    - Automatically switches between replicas

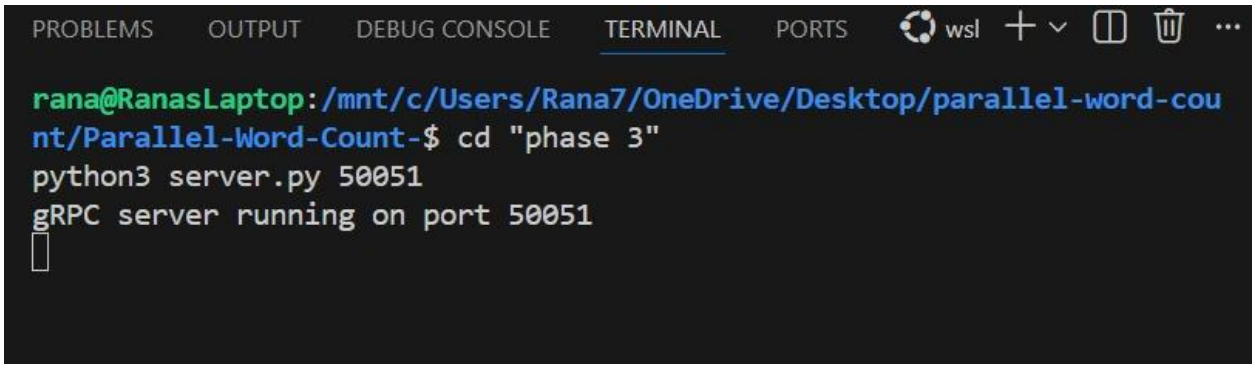
Each server instance processes requests independently and executes the word count logic using the MPI-based backend.

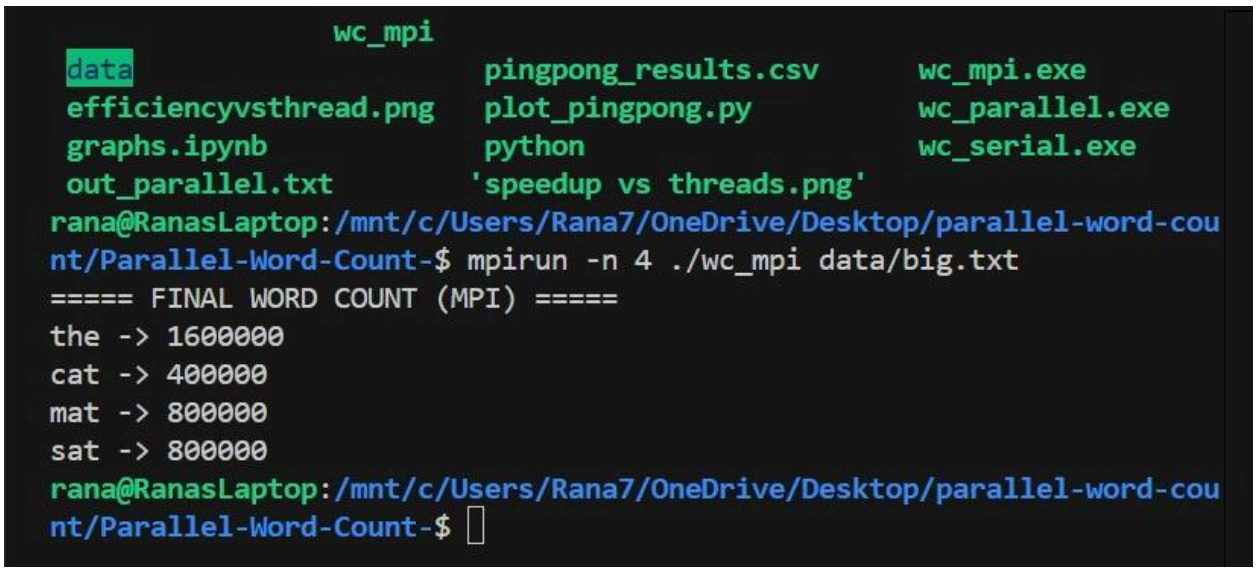
---

3. Replication Strategy

Replication is implemented by running multiple instances of the same gRPC service on different ports. Both replicas expose **identical service interfaces** and can serve client requests interchangeably.


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
wsl
rana@RanasLaptop:/mnt/c/Users/Rana7/OneDrive/Desktop/parallel-word-cou
nt/Parallel-Word-Count-$ cd "phase 3"
python3 server.py 50052
gRPC server running on port 50052


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
wsl
rana@RanasLaptop:/mnt/c/Users/Rana7/OneDrive/Desktop/parallel-word-cou
nt/Parallel-Word-Count-$ cd "phase 3"
python3 server.py 50051
gRPC server running on port 50051


wc_mpi
data
pingpong_results.csv
wc_mpi.exe
efficiencyvsthread.png
plot_pingpong.py
wc_parallel.exe
graphs.ipynb
python
wc_serial.exe
out_parallel.txt
'speedup vs threads.png'
rana@RanasLaptop:/mnt/c/Users/Rana7/OneDrive/Desktop/parallel-word-cou
nt/Parallel-Word-Count-$ mpirun -n 4 ./wc_mpi data/big.txt
===== FINAL WORD COUNT (MPI) =====
the -> 1600000
cat -> 400000
mat -> 800000
sat -> 800000
rana@RanasLaptop:/mnt/c/Users/Rana7/OneDrive/Desktop/parallel-word-cou
nt/Parallel-Word-Count-$

4. Fault Tolerance Mechanisms

4.1 Client-side Retries

The client implements a **retry mechanism** to ensure robust request handling:

- Each request is retried up to a fixed number of attempts.
- If one replica fails, the client automatically switches to the next available replica.
- No manual intervention is required.

This mechanism allows the system to tolerate **transient failures** and service crashes, maintaining continuous operation without human supervision.

4.2 Idempotency Support

Each request is assigned a unique request_id. The server maintains an in-memory cache of processed requests to ensure idempotent behavior.

- Duplicate requests resulting from client retries are not reprocessed.
- This improves overall system reliability and prevents inconsistent results during failover events.

---

5. Fault Injection and Observed Failures

5.1 Failure Type 1: (Optional/Other failures if applicable)

5.2 Failure Type 2: Service Execution Failure

Injection Method:
The gRPC server attempted to execute the MPI-based word count backend, which resulted in an execution failure due to **platform incompatibility** between the execution environment and the binary format.

Observed Behavior:

- The server returned **internal errors** during request processing.
- The gRPC service itself **remained alive**.
- The client retried requests across available replicas automatically.
- The system did **not crash or deadlock**.

Result:
The system demonstrated **resilience against execution-level service failures**, maintaining stability and controlled error handling.

---

6. Experimental Evidence

To evaluate fault tolerance, the following artifacts were collected:

- **Terminal logs** showing replica failover events.
- **Client retry outputs**, documenting automatic switching between replicas.
- test_metrics.csv, containing timestamps, replica attempts, request success status, and latency measurements.

These logs provide clear evidence of:

- **Automatic recovery behavior**
- **Fault-aware request handling**
- **System stability under service execution failures**

phase 3 > test_metrics.csv > data

```csv
ts,replica,success,latency_ms,input_text
1766768085.2094405,localhost:50051,False,872.9314804077148,hello world hello
1766768086.6049492,localhost:50051,False,894.9525356292725,hello world hello
1766768087.9381344,localhost:50051,False,832.2796821594238,hello world hello
1766768089.3146312,localhost:50051,False,875.9109973907471,hello world hello
1766768090.7079086,localhost:50051,False,892.5347328186035,hello world hello
1766768092.0692067,localhost:50051,False,860.5849742889404,hello world hello
1766768093.4941187,localhost:50051,False,924.1678714752197,hello world hello
1766768094.9708526,localhost:50051,False,976.0546684265137,hello world hello
1766768096.3742397,localhost:50051,False,902.5745391845703,hello world hello
1766768097.8372042,localhost:50051,False,962.6617431640625,hello world hello

```

Real-Time Word Count System with gRPC and Spark Structured Streaming

- **Overview**
  Microservice-based architectures, distributed computation, and real-time processing are becoming more and more important in today's data-intensive systems. Applications that need low latency, scalability, and continuous data ingestion can no longer rely on traditional batch processing. A real-time data processing pipeline utilizing Apache Spark Structured Streaming combined with a gRPC-based microservice is fully implemented in this project.

- **Problem Description**
  Inter-service communication must be extremely dependable, scalable processing should be modular in design, and real-time applications require constant ingestion. Flexibility and maintainability are decreased when business logic is directly integrated into streaming engines. This project's solution uses gRPC microservices to decouple the processing logic.

- **System Overview**
  The system consists of:
- A streaming data generator
- A Spark Structured Streaming application
- A gRPC Word Count microservice

4. **What Was Implemented**
- Micro-batch streaming pipeline using Spark
- gRPC integration via Spark UDFs
- Real-time aggregation of results
- Fault tolerance using checkpointing

5. **Importance of This Work**
   This implementation demonstrates a production-style architecture for real-time systems, highlighting clean separation of concerns and scalable design.
6. **Benefits**

- Scalability
- Modularity
- Fault tolerance
- Real-time insights
- Production readiness

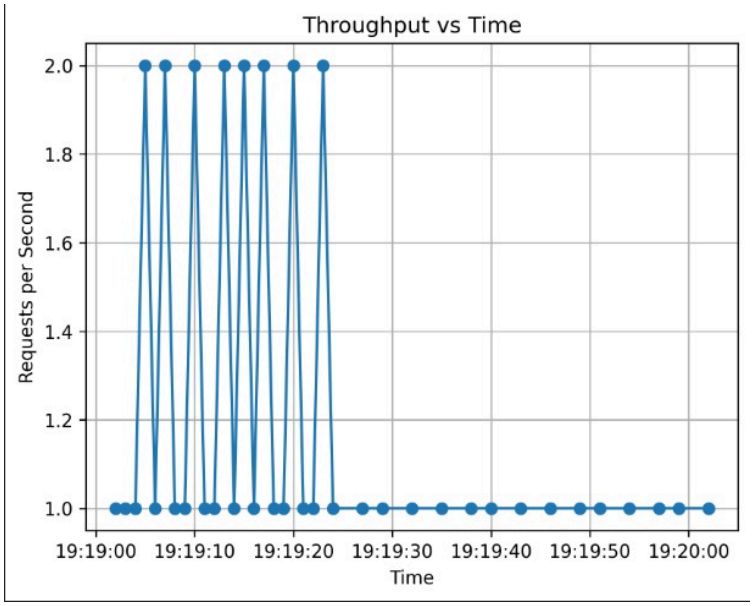
7. **Applications**
   The system can be extended for NLP pipelines, ML inference, log processing, and real-time analytics.

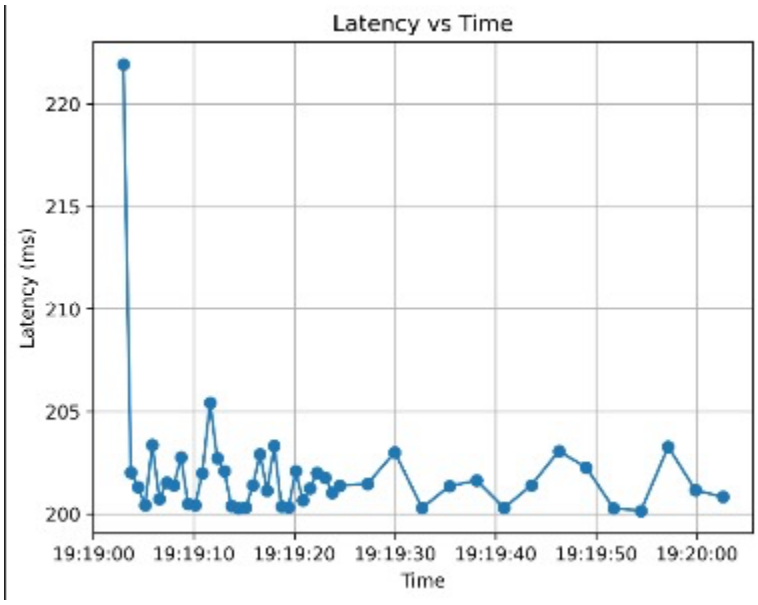
Performance Metrics Summary
The following table summarizes the key performance metrics computed from the experimental

Performance Metrics

Metric	Value
Average Latency (ms)	201.52
Throughput (requests/sec)	0.75
Recovery Time (sec)	0.75

Analysis
The system achieved an **average latency of 201.52 ms**, indicating stable response times during steady-state operation after excluding the warm-up phase.
The observed **throughput of 0.75 requests per second** reflects consistent request processing under the given workload and retry mechanism.
Following a replica failure, the system demonstrated fast recovery, with a **recovery time of 0.75 seconds**, confirming effective fault tolerance and rapid redirection of requests to the remaining active replica.


Throughput vs Time
Requests per Second
2.0
1.8
1.6
1.4
1.2
1.0
19:19:00
19:19:10
19:19:20
19:19:30
19:19:40
19:19:50
19:20:00
Time


Latency vs Time
Latency (ms)
220
215
210
205
200
19:19:00
19:19:10
19:19:20
19:19:30
19:19:40
19:19:50
19:20:00
Time